Tower of Hanoi Problem

It is widely known as a children's game. The Tower of Hanoi is a classic mathematical puzzle invented by the French mathematician Édouard Lucas. It is also called the Tower of Brahma or Lucas' Tower. It consists of three rods and a number of disks of different sizes that are initially arranged in ascending order on one rod, with the smallest disk on top. It is widely used to illustrate the concept of recursion in computer science.

Core Components

Rod / Peg / Tower / Pole / Pillar: The three vertical rods where disks are placed. They are usually named source, auxiliary (temp/helper), and destination.

Disks: Circular plates of different sizes.

Initial State: All disks are stacked on the source rod in decreasing order of diameter, with the largest one at the bottom and the smallest one at the top.

Goal State: All disks are moved from the source rod to the destination rod using the auxiliary rod as an intermediate.

Rules/Conditions to be Followed:

1. Move only one disk at a time from one rod to another.
2. At each step, we move/take out the topmost disk from one rod and place it onto another rod.
3. A larger disk can never be placed on top of a smaller disk.
4. An auxiliary rod can be used as temporary storage or a helper while moving disks.

Key Concept

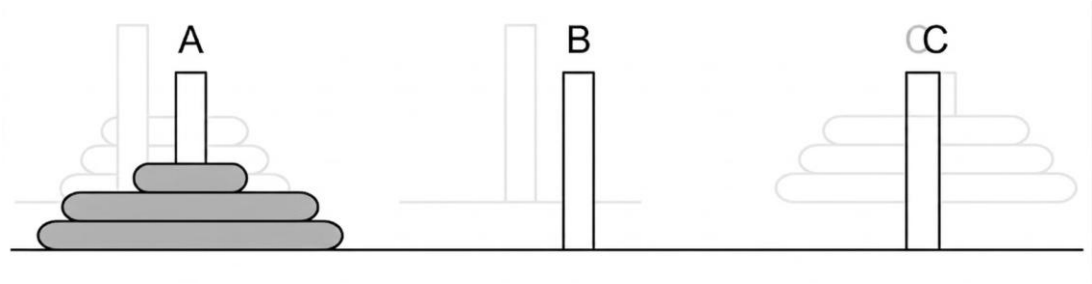
Minimal Moves: For n disks, the minimum number of moves/ steps required is 2^n-1.

Recursion Concept: The solution uses a recursive algorithm, which can be described as follows:

1. Move the top n-1 disks from the source rod to the auxiliary rod using the destination rod as auxiliary.
2. Move the remaining disk from the source rod to the destination rod.
3. Move the n-1 disks from the auxiliary rod to the destination rod using the source rod as auxiliary.

Example:

The solution to the 3-disks Towers of Hanoi problem as:



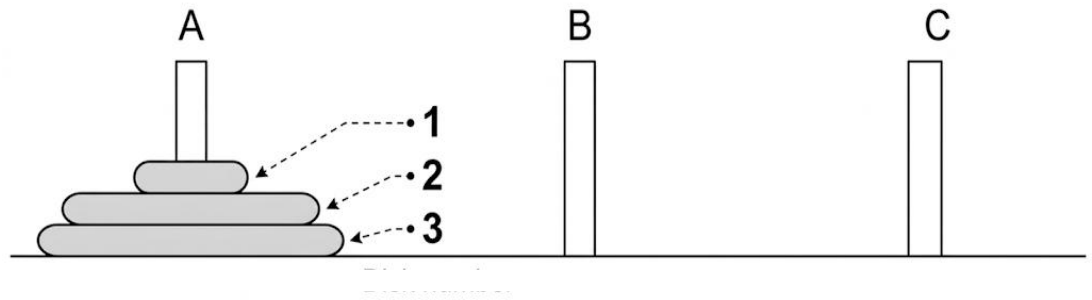
Initially, suppose there are three pegs labeled A, B, and C. Three disks of different sizes are placed on peg A. The arrangement of the disks is done in such way that every disk is placed on the larger disk.

The objective of the problem is to move all three disks from peg A to peg C using peg B as an auxiliary.

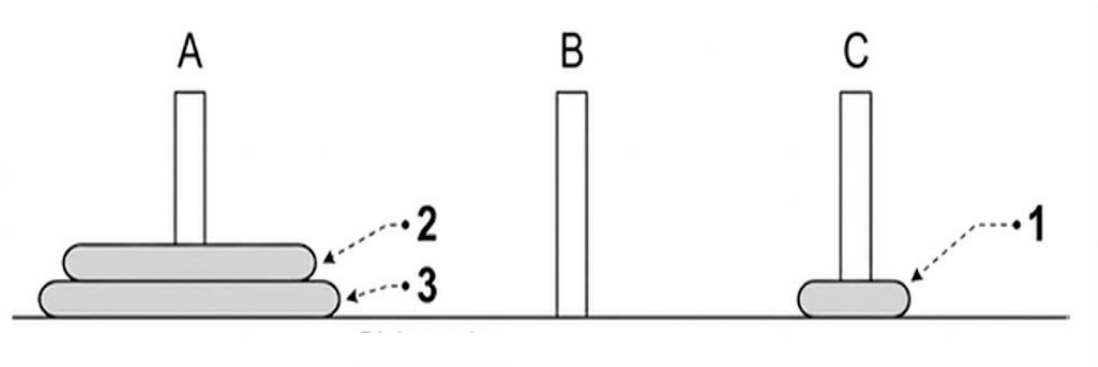
The conditions are :

1. Only the top disk on any peg may be moved at a time.
2. A larger disk should never be placed on a smaller disk.

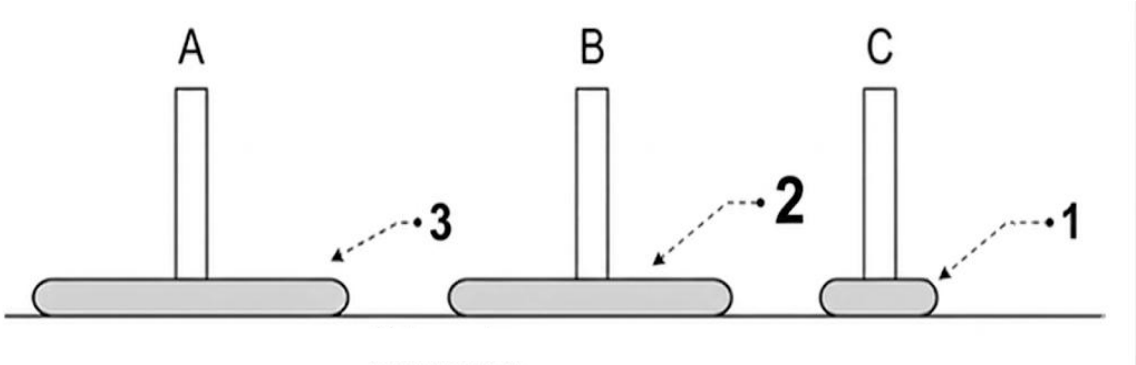
For convenience, let us number the disks from top to bottom as:



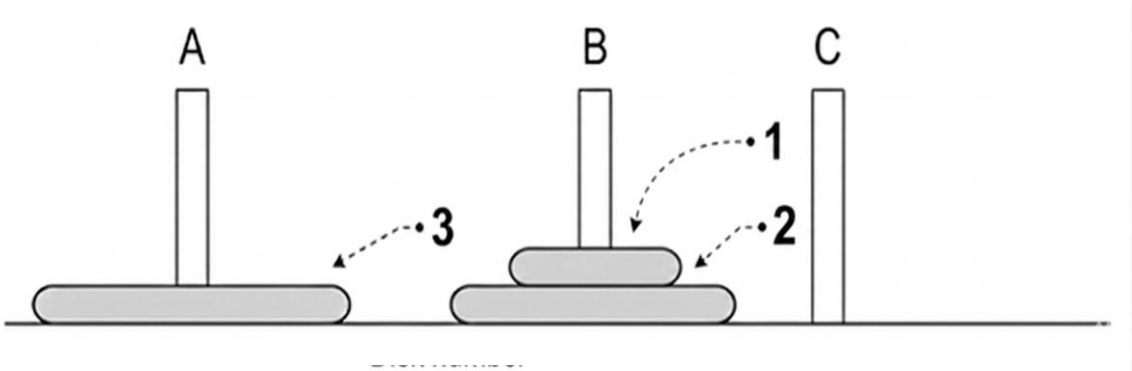
Step 1: Move disk 1 from A to C using B as auxiliary.



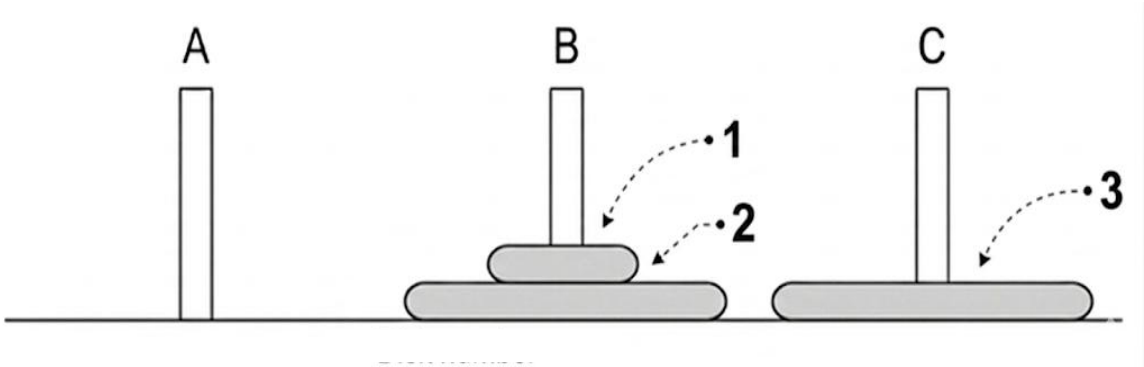
Step 2: Move disk 2 from A to B using C as auxiliary.



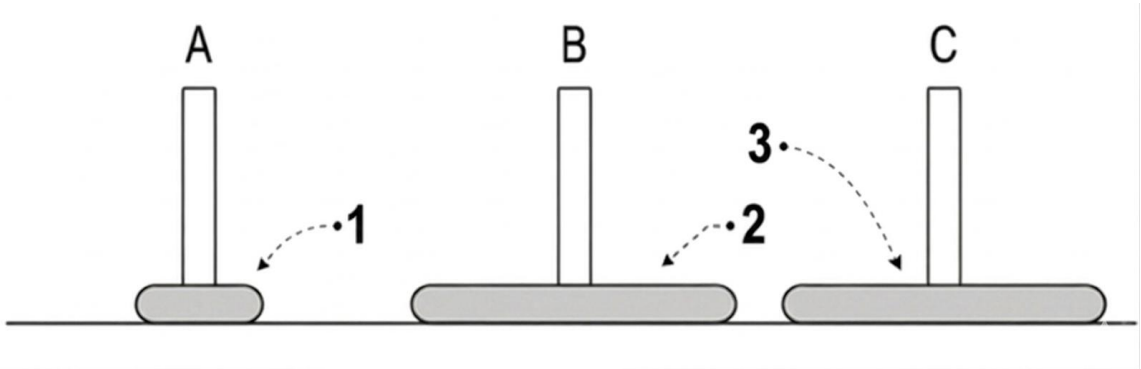
Step 3: Move disk 1 from C to B using A as auxiliary.



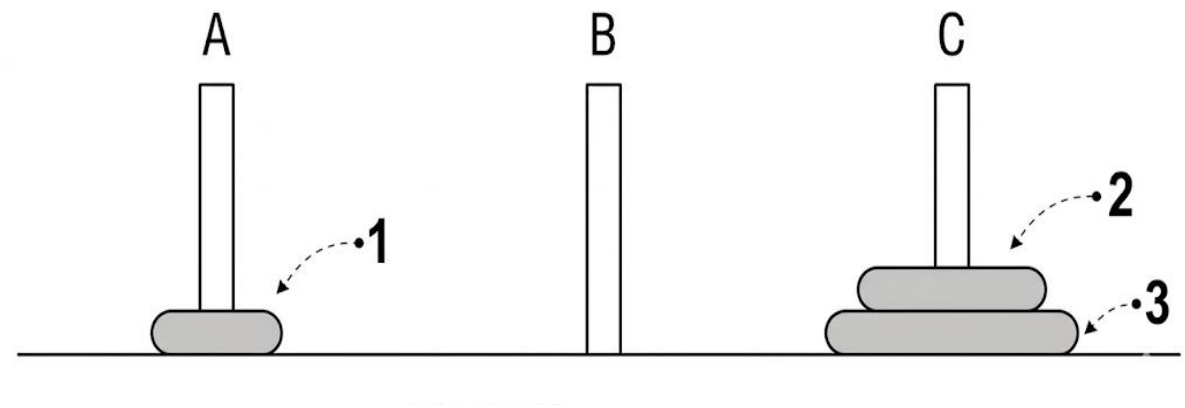
Step 4: Move disk 3 from A to C.



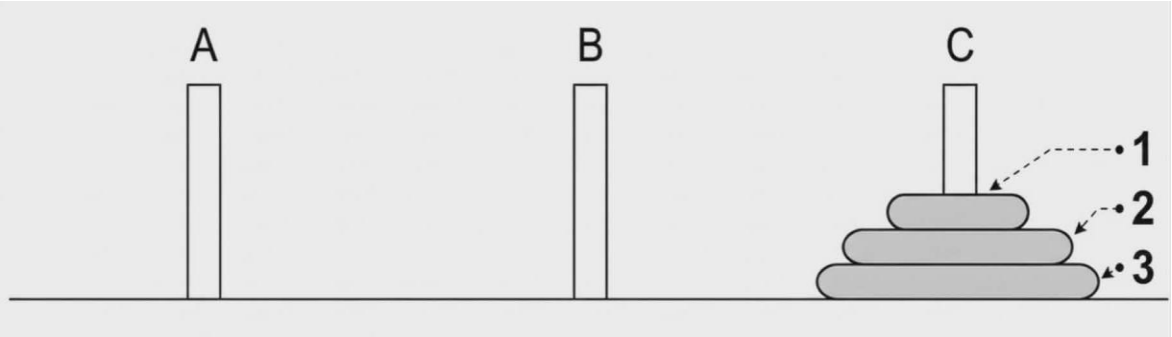
Step 5: Move disk 1 from B to A.



Step 6: Move disk 2 from B to C.



Step 7: Move disk 1 from A to C.



Hence, total number of moves when n=3,
Number of moves/ steps required=2^n-1=2^3-1=7

Algorithm Step:

```
TOH(n, src, aux, dest)
   // base case: only one disk to move
   if n == 1
      print "Move disk", n, "from", src, "to", dest
   else
      // recursive case 1: move n-1 disks from source to auxiliary
      TOH(n-1, src, dest, aux)s
      print "Move disk", n, "from", src, "to", dest
      // recursive case 2: move n-1 disks from auxiliary to destination
      TOH(n-1, aux, src, dest)
End
```

Note:

Base case can be taken as:

- n==1
- n==0
- n<=0

Recursive Trace for n Disks

```
TOH(n, src, aux, dest)
  if n == 1
    print src -> dest
  else
    // Move n-1 disks from src to aux
    TOH(n-1, src, dest, aux)
    print src -> dest
    // Move n-1 disks from aux to dest
    TOH(n-1, aux, src, dest)
```

For n=3, we traced the recursive the tree as :

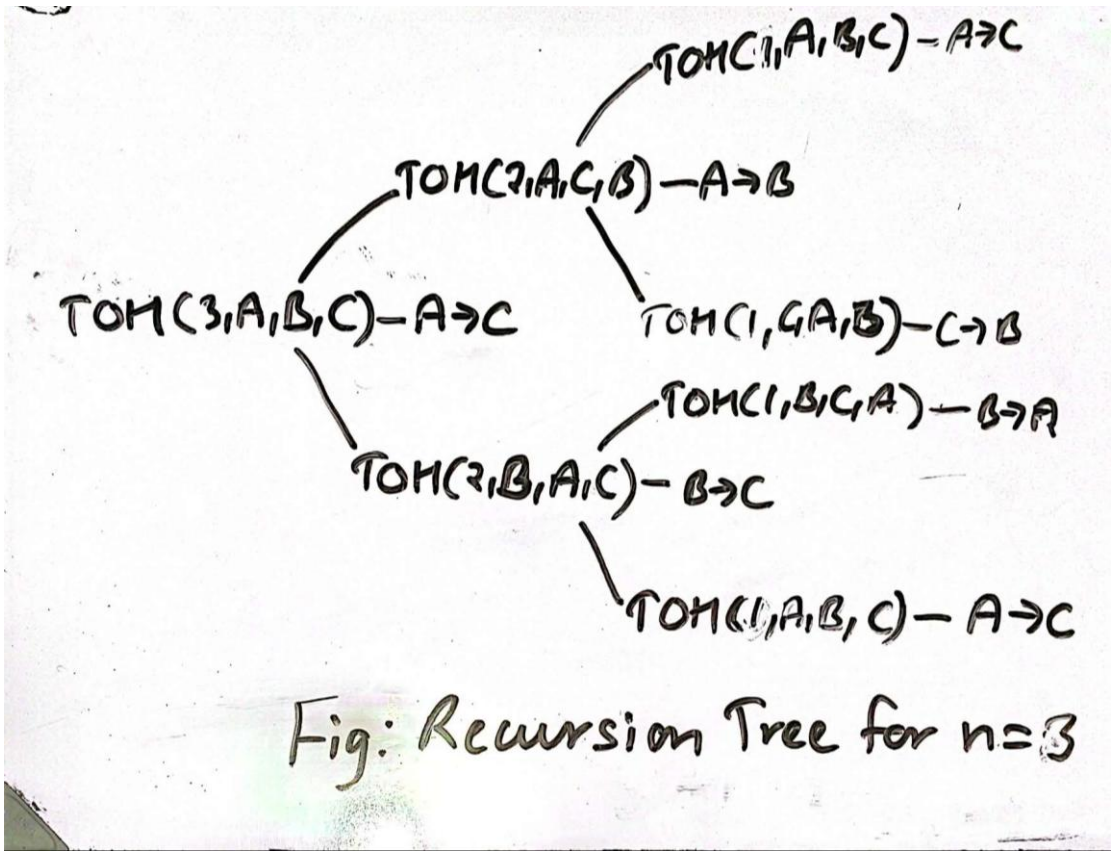


Fig: Recursion Tree for n=3

The sequence of moves is as follows:

1. A->C
2. A->B
3. C->B
4. A->C
5. B->A
6. B->C
7. A->C

Trick: TOH(n, A, B, C)

1. Make A → C
2. Swap B and C
   i.e. TOH(n−1, A, C, B)
3. Make A → B
4. Swap A and B
   i.e. TOH(n−1, B, A, C)
5. Repeat the above steps until n == 1

Analysis of TOH Algo.:

- **Time Complexity:** $O(2^n)$
- **Space Complexity:** $O(n)$